Pashov Audit Group

Regnum Aurum Security Review

November 4th 2025 - November 5th 2025

Contents

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Regnum Aurum

Regnum Aurum (RAAC) is a fractionalization platform that tokenizes real estate into NFTs (RAACNFT) and fractional index tokens (iRAAC), enabling on-chain lending, borrowing, and liquidity against property value. By combining Chainlink-powered appraisals, a hybrid RWA Vault, and veRAAC governance the protocol enables programmable debt positions against real estate assets with on-chain liquidation mechanisms.

5. Executive Summary

A time-boxed security review of the **RegnumAurumAcquisitionCorp/contracts** repository was done by Pashov Audit Group, during which **Shaka**, **0xTheBlackPanther**, **0xdeadbeef** engaged to review **Regnum Aurum**. A total of **6** issues were uncovered.

Protocol Summary	
Project Name	Regnum Aurum
Protocol Type	RWA tokenization
Timeline	November 4th 2025 - November 5th 2025

Review commit hash:
- 9846f571615b7a51d815fbc8841300ca75780a70 (RegnumAurumAcquisitionCorp/contracts)

Fixes review commit hash:
- 486262a135c0405f5f18e20bf042ab256082686c (RegnumAurumAcquisitionCorp/contracts)

Scope

`RWAIndexTokenOracle.sol` `LendingPool.sol` `LendingPoolStorage.solLiquidationSwap.sol` `StabilityPool.sol` `DEToken.sol` `RAACNFT.sol` `RToken.sol` `RAACNFTVaultAdapterOracle.sol` `RAACNFTVaultAdapterV2.sol` `RWAVault.sol` `ILendingPool.sol` `IDEToken.sol` `IRToken.sol` `WadRayMath.sol`

6. Findings

Findings count

Severity	Amount
Low	6
Total findings	**6**

Summary of findings

ID	Title	Severity	Status
[L-01]	Position scaled debt can be underestimated	Low	**Resolved**
[L-02]	Missing setter function for mutable oracle address in `RWAIndexTokenOracle`	Low	**Resolved**
[L-03]	Incorrect comment in `LiquiditySwap`	Low	**Resolved**
[L-04]	`rateData.maxRate` incorrect comment	Low	**Resolved**
[L-05]	Liquidations occur during paused `lendingPool` preventing fair repayment	Low	Acknowledged
[L-06]	`LiquidationSwap` lacks constructor-set oracle	Low	**Resolved**

Low findings

[L-01] Position scaled debt can be underestimated

`LendingPool._positionScaledDebt()` calculates the scaled debt for a given position by multiplying the raw debt balance by the index multiplier. This operation has been updated to round down instead of rounding to the nearest value.

Generally, it should be avoided to underestimate debt values so that lenders and the protocol are never exposed to unexpected losses.

It is recommended to round up or at least round to the nearest value when calculating debt amounts.

[L-02] Missing setter function for mutable oracle address in `RWAIndexTokenOracle`

In `contracts/contracts/core/oracles/RWAIndexTokenOracle.sol` the `crvusdToUSDOracle` state variable is declared as mutable (*non-immutable*) but lacks a setter function to update it

```
address public crvusdToUSDOracle; // Mutable but no setter @audit
```

This is problematic if the crvUSD oracle is compromised, deprecated, or contains bugs, there is no way to update the reference without redeploying all dependent contracts.

Recommendation

Add an owner-controlled setter function with proper access control and validation.

```
event CrvUSDOracleUpdated(address indexed oldOracle, address indexed newOracle);

function setCrvUSDOracle(address _newOracle) external onlyOwner {
    require(_newOracle != address(0), "Zero address");
    require(_newOracle != crvusdToUSDOracle, "Same oracle");

    address oldOracle = crvusdToUSDOracle;
    crvusdToUSDOracle = _newOracle;

    emit CrvUSDOracleUpdated(oldOracle, _newOracle);
}
```

Of if the oracle should never change, make it immutable.

```
address public immutable crvusdToUSDOracle;
```

[L-03] Incorrect comment in `LiquiditySwap`

The following comment has not been changed, even though the use of an oracle to convert `crvUSD` to `USD` is in place:

```
// Price per share returns 1e18 price of the iRAAC in crvUSD.
```

[L-04] `rateData.maxRate` incorrect comment

`rateData.maxRate` is set to `83.63` although comment suggests `83.75`:

```
83.75% in RAY (27 decimals) based on 7.25% US Prime Rate
```

[L-05] Liquidations occur during paused `lendingPool` preventing fair repayment

`StabilityPool.liquidateBorrower` can be called if the underlying `LendingPool` is paused. Because repayment in `LendingPool` is gated by `whenNotPaused`, pausing the `LendingPool` prevents borrowers from repaying or closing liquidation within the grace period, while liquidation can still be initiated/finalized from the `StabilityPool` side. This creates an unfair situation: borrowers cannot cure their positions, yet their collateral can still be liquidated.

Recommendations

Prevent liquidation initiation/finalization when the LendingPool is paused.

[L-06] `LiquidationSwap` lacks constructor-set oracle

`LiquidationSwap` uses the external `crvusdToUSDOracle` but does not require the oracle to be set in the constructor.

The contract provides `setCrvusdToUSDOracle(address)` but the constructor does not initialize it. `_swap()` unconditionally calls `ICrvUSDToUSDOracle(crvusdToUSDOracle).getPrice()`.

If the oracle address is unset or misconfigured, the call reverts, causing a DOS.

Impact: Liquidations can be blocked.

Recommendations

Add oracle on contract construction.